Data Preprocessing and Visualization

Load data • Clean & transform • Aggregate & group • Visualize insights

pandas

From raw data to meaningful insights!

1. Data Loading, Storage, and File Formats

- Reading and writing data in text format
- Binary data formats (e.g., pickle, parquet)
- Interacting with HTML and web APIs
- Interacting with databases

CSV JSON HTML DB Pickle Parquet

1.1 Read and Write Text Files

Text file (CSV)

```
import pandas as pd
df = pd.read_csv('data.csv')   # read
df.to_csv('output.csv', index=False)   # write
```

1.2 Read Binary Format

Binary file (Parquet)

```
import pandas as pd
df = pd.read_parquet('data.parquet')
```

1.3 Interact with Web API (JSON)

Web API (JSON)

```
import requests
url = 'https://api.example.com/data'
response = requests.get(url).json()
print(response)
```

JSON

1.4 Connect to Database

Database (SQL)

```
import sqlite3
conn = sqlite3.connect('data.db')
df = pd.read_sql('SELECT * FROM table', conn)
print.close()
```

2. Data Wrangling: Clean, Transform, Merge, Reshape

- Combining and merging data sets
- Reshaping and pivoting
- Data transformation
- String manipulation

Clean → Transform → Merge → Reshape

2.1 Combining and Merging Data Sets

Merge data sets

```
import pandas as pd
df1 = pd.read_csv('sales1.csv')
df2 = pd.read_csv('sales2.csv')
merged = pd.merge(df1, df2, on='id')
print(merged.head())
```

id	name	sales
1	Alice	100
2	Bob	150
3	Charlie	200

2.2 Reshaping and Pivoting

Pivot table (reshape data)

```
pivot_df = df.pivot_table(
    index='category',
    columns='month',
    values='sales', aggfunc='sum')
print(pivot_df)
```

Month	Jan	Feb	Mar
A	100	120	130
B	200	220	250

2.3 Data Transformation

Transform data (types, values)

```
df['price'] = df['price'].apply(lambda x: x*1.1)
df['date'] = pd.to_datetime(df['date'])
df['year'] = df['date'].dt.year
```

2.4 String Manipulation

Work with text data

```
df['name'] = df['name'].str.strip()
df['name'] = df['name'].str.lower()
df['email'] = df['email'].str.split('@').str[0]
```

3. Data Aggregation and Group Operations

- Group by Mechanics
- Data Aggregation
- Groupby Operations and Transformations
- Pivot Tables and Cross Tabulation

Group by → Aggregate & Analyze

3.1 Group By Mechanics

Group data by category

```
grouped = df.groupby('department')
print(grouped.size())
```

department	count
HR	10
IT	15
Sales	12

3.2 Data Aggregation

Compute statistics

```
agg_df = df.groupby('department')[
  'salary'].agg(['mean','sum'])
print(agg_df)
```

department	mean	sum
HR	50000	500000
IT	70000	1050000
Sales	60000	720000

3.3 Groupby Transformations

Apply transformations

```
df['rank'] = df.groupby('department')['salary'] \
   .rank(ascending=False)
print(df[['department','salary','rank']])
```

3.4 Pivot Tables and Cross Tabulation

Pivot / Crosstab

```
pivot = pd.pivot_table(df, values='sales',\
index='region', columns='product', aggfunc='sum')
print(pivot)
```

Product	A	B	C
North	100	200	150
South	120	180	160

4. Data Visualization

- A Brief matplotlib API Primer
- Statistical Visualization with Seaborn
- Plotting Functions in pandas
- Time Series
- Real-world case study applications

matplotlib seaborn pandas

Turn data into insights!

4.1 Matplotlib Primer

Basic plotting with matplotlib

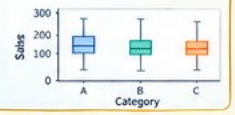
```
import matplotlib.pyplot as plt
plt.plot(df['date'], df['sales'])
plt.title('Sales Trend')
plt.show()
```



4.2 Seaborn Visualization

Statistical plots (with seaborn)

```
import seaborn as sns
sns.boxplot(x='category',y='sales',data=df)
plt.show()
```



4.3 Plotting Functions in pandas

Quick plots with pandas

```
df.plot(kind='bar', x='product', y='sales')
plt.show()
```



4.4 Time Series

Time-based analysis

```
df.set_index('date', inplace=True)
df.resample('M').mean().plot()
plt.show()
```



4.5 Case Study

- Sales analysis
- Customer behavior
- Forecasting
- Business insights

Real-world application

Load → Clean → Transform → Analyze → Visualize → Get Insights

= Better data Better decisions!

Data Preprocessing and Visualization:

Data Loading, Storage

From raw data to meaningful insights!

Let's turn raw data into clean, useful and visual insights!

We will load data, preprocess it, store it and visualize it using Python!

1. What is Data Preprocessing?

It's the process of cleaning and transforming raw data into a usable and high-quality format.

- Handle missing values
- Remove duplicates
- Convert data types
- Normalize / scale data
- Fix inconsistent data

2. Loading Data

Bring data into Python using Pandas:

- CSV: pd.read_csv()
- Excel: pd.read_excel()
- JSON: pd.read_json()
- SQL: pd.read_sql()

3. Exploring Data

Get a quick understanding of your data:

- df.head() – first rows
- df.info() – data types
- df.describe() – summary
- df.shape() – dimensions

4. Handling Missing Data

Use effective methods:

- dropna() – remove rows/cols with missing values
- fillna() – fill with a value (mean, median, etc.)
- interpolate() – estimate missing values

Age	Salary
25	50000
NaN	60000
30	70000

5. Data Transformation

Prepare and convert data for better analysis:

- astype() – change data type
- apply() – custom function
- map() – replace values
- normalize() – scale values
- encode() – convert categories

6. Data Storage

Save your processed data efficiently:

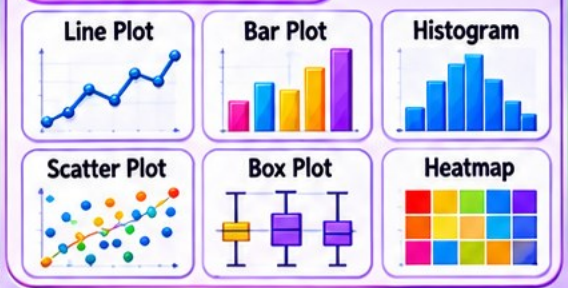
- CSV: df.to_csv()
- Excel: df.to_excel()
- Parquet: df.to_parquet()
- Pickle: df.to_pickle()

7. Data Visualization

Create meaningful visuals from data using:

- Matplotlib – basic & flexible
- Seaborn – beautiful & statistical
- Plotly – interactive & modern

8. Common Plots



9. Best Practices

- Check and clean data quality
- Choose the right storage format (CSV, Excel, Parquet, Pickle)
- Use appropriate visualizations for your goal
- Keep code modular and reusable
- Document your data workflow

Clean Data + Organized Storage + Clear Visualization = Better Insights!

Key Takeaway:

Preprocess data effectively, store it smartly, and visualize it clearly to make better, data-driven decisions!

File Formats

Reading and Writing data in text format, binary data formats

Files store data in different formats. Let's explore them!

Python makes it easy to read and write data in many formats!

1. What are File Formats?

File formats define how data is stored in a file — as text or binary.



2. Text Formats

Easy to read, human-readable formats.



.txt



.csv



.xlsx



.json

3. Binary Formats

Compact and efficient formats for data.



.bin



.pkl



.npy



.parquet

4. Reading Text Files

Use built-in functions to read text data.

```
with open('data.txt', 'r') as f:
    content = f.read()
    print(content)
```

Read text files easily!

5. Writing Text Files

Use write() to save data in text format.

```
with open('output.txt', 'w') as f:
    f.write('Hello, Python!')
```

Save your results!

6. Working with Binary Files

Use binary mode for binary data.

```
with open('data.pkl', 'rb') as f:
    data = pickle.load(f)
```

```
with open('data.pkl', 'wb') as f:
    pickle.dump(data, f)
```

Read & Write Binary Data!

7. Popular Formats in Pandas

Read and write structured data easily.

```
import pandas as pd

df = pd.read_csv('data.csv')
df.to.excel('data.xlsx', index=False)
```

CSV, Excel, JSON & More!

8. Key Takeaways

- Text formats: .txt, .csv, .json
- Binary formats: .pkl, .npy, .parquet
- Use open(), read() and write()
- Pandas helps with CSV, Excel, JSON
- Choose the right format for your needs!

Explore Different File Formats and Make Your Data Work Better!


Let's explore web technologies and databases with Python!
Interacting with HTML and Web APIs, Interacting with Databases
Fetch • Parse • Store • Analyze
Get data from the web and store it in databases!
1. Working with HTML
Use Python to interact with HTML content.
from bs4 import BeautifulSoup
soup = BeautifulSoup(html, 'html.parser')
print(soup.title.text)
HTML
Parse HTML
Extract useful data
2. Consuming Web APIs
Fetch data from web APIs using requests.
import requests
response = requests.get('https://api.example.com')
data = response.json()
API
{...}
Get data (JSON)
Use API endpoints
3. Processing Web Data
Work with JSON and convert it to data frames.
import pandas as pd
df = pd.json_normalize(data)
print(df.head())
JSON
Handle JSON
Analyze data
4. Interacting with Databases
Connect and query databases using SQL and SQLAlchemy.
import sqlite3
conn = sqlite3.connect('data.db')
df = pd.read_sql_query('SELECT * FROM users', conn)
Read & write data
Use SQL queries
5. Popular Databases
Work with different databases.
SQLite
MySQL
PostgreSQL
SQL Server
6. Advanced Operations
Perform more database operations.
Insert, Update, Delete
Filter and Sort data
Use pandas with SQL
7. Real-World Uses
Apply these skills in real projects.
Web Scraping
Data Dashboards
Data Analysis
Web Apps
8. Key Takeaways
Use requests to work with web APIs
Parse HTML with BeautifulSoup
Interact with databases using SQL & pandas
Build powerful data-driven applications
Explore. Extract. Store. Grow!
From the Web to Your Database – Python Makes It Possible!

Data Wrangling: Clean, Transform

Turn Messy Data into Meaningful Insights!



What is Data Wrangling?

It's the process of cleaning and transforming raw data into a clean, structured format for analysis.

Step 1: Clean Your Data



Examples:

- Remove duplicates
- Handle missing values
- Correct data types

Step 2: Transform Your Data



Examples:

- Normalize values
- Convert formats
- Create new columns

Python Tools

Use powerful Pandas functions to clean and transform data.

```
import pandas as pd
df = pd.read_csv('data.csv')
df.drop_duplicates(inplace=True)
df['date'] = pd.to_datetime(df['date'])
df['price'] = df['price'].astype(float)
```



Common Operations

- Remove duplicates
- Handle missing data
- Standardize formats
- Merge datasets
- Create new features
- Filter and sort data



Real-World Example

Imagine working with customer data from different sources...

CSV

Database

→ Clean + Transform = Ready for Analysis!

Key Takeaways

- Data wrangling = Clean + Transform
- Use Pandas for easy data handling
- Handle missing and duplicate data
- Reshape data for better insights
- Clean data leads to better decisions!





Merge, Reshape

Combining and Merging Data Sets, Reshaping and Pivoting



1. Merge – Combine Data

Join datasets using common keys.

```
pd.merge(df1, df2, on='id')
```

df1 + df2 → Merged Data

2. Types of Merges

Choose the right join:

Inner	Left	Right	Outer
Common rows	All from left	All from right	All rows

3. Reshape Data

Change the structure of your data.

Melt (Wide → Long)

```
pd.melt(df,
id_vars=['id'])
```

Pivot (Long → Wide)

```
df.pivot(index='id',
columns='type',
values='value')
```



4. Pivot Tables

Summarize data with pivot tables.

```
df.pivot_table(values='sales',
      index='product',
      aggfunc='sum')
```



5. Common Operations

Merge, reshape and analyze data easily.

- Concatenate: `pd.concat()`
- Reshape: `df.stack()`, `df.unstack()`
- Pivot: `df.pivot()`, `df.pivot_table()`



6. Real-World Example

Combine monthly sales and reshape for analysis.

Sales Jan	
Product	Sales
A	100
B	150

+

Sales Feb	
Product	Sales
A	120
B	180

→

Final Data		
Product	Jan	Feb
A	100	120
B	150	180

7. Key Takeaways

- Merge datasets using keys.
- Reshape data with melt, pivot.
- Use pivot tables for summaries.
- Organize data for deeper insights.

Combine • Reshape • Analyze



Data Transformation & String Manipulation

Clean, Convert, and Work with Text Data!

Data is often in different formats. Let's transform it and handle strings efficiently!

Exactly! Data Transformation helps us make data usable, and string manipulation lets us work with text!

1. Data Transformation

What do you mean by data transformation?

It means converting data into a more useful format — like changing data types, scaling values, or creating new features.

2. Common Transformations

- Convert data types: astype()
- Scale data: normalize(), standardize()
- Create new columns: apply(), map()
- Handle dates: to_datetime()

Example:

```
df['age'] = df['age'].astype(int)
df['score'] = df['score'].round(2)
```

3. String Manipulation

Strings are everywhere! Let's see how to manipulate them.

- Change case: str.lower(), str.upper()
- Trim spaces: str.strip()
- Replace text: str.replace()
- Split & Join: str.split(), str.join()

4. Examples

Convert to lowercase:

```
df['name'] = df['name'].str.lower()
```

Replace text:

```
df['city'] = df['city'].str.replace(
  'New', 'N')
```

Split and get first part:

```
df['email'] = df['email'].str.split('@').str[0]
```

5. More Transformations

- Apply functions: `df['col'].apply(lambda x: x*2)`
- Map values: `df['grade'].map({'A':1,'B':2})`
- Use replace: `df['text'].str.replace('-', ' ')`

6. Real-World Use

Data Transformation and String Manipulation help in:

- Cleaning customer data
- Processing text reviews
- Preparing data for analysis
- Building reports & dashboards

Clean data leads to better insights!

7. Key Takeaways

- Data Transformation makes data usable.
- String Manipulation helps you work with text.
- Use pandas and Python string methods effectively.
- Clean, convert and analyze with confidence!

Transform Your Data, Unlock New Insights!

Master these skills and make your data truly powerful!

Data Aggregation and Group Operations

Group by Mechanics, Data Aggregation

We often have data with multiple categories. How do we summarize it?

That's where groupby() comes in! It lets us group data and apply useful functions.

1. What is GroupBy?

GroupBy splits data into groups based on a key column.

```
df.groupby('Category')
```

It helps us analyze data by groups!

2. Common Aggregation Functions

Use functions to summarize each group.

- sum() – Total
- mean() – Average
- count() – Count
- min() – Minimum
- max() – Maximum

Category	Sales
A	100
B	150
A	200
B	120

3. Examples

Get useful insights with groupby!

```
df.groupby('Category')['Sales'].sum()

df.groupby('Category')['Sales'].mean()

df.groupby('Category').agg({
    'Sales': ['sum', 'mean'],
    'Profit': 'max'
})
```

You can also use multiple aggregations!

4. GroupBy with Multiple Columns

Group by more than one column.

```
df.groupby(['Region', 'Category'])
  ['Sales'].sum()
```

Region	Category	Sales
East	A	100
East	B	120
West	A	150
West	B	180

5. Advanced Operations

Try more powerful options!

- filter() – Filter groups
- transform() – Modify data
- apply() – Custom functions

These help us get deeper insights!

6. Real-World Use Cases

- Sales analysis by region
- Customer segmentation
- Performance reports
- Data dashboards



7. Key Takeaways

- GroupBy helps summarize data by groups.
- Use aggregation functions (sum, mean, count...).
- Combine with multiple columns for deeper analysis.
- Turn data into actionable insights!

Group. Aggregate. Discover!

Data Aggregation and Group Operations

Group by Mechanics, Data Aggregation

We often have data with multiple categories. How do we summarize it?

That's where groupby() comes in! It lets us group data and apply useful functions.

1. What is GroupBy?

GroupBy splits data into groups based on a key column.

```
df.groupby('Category')
```

It helps us analyze data by groups!

2. Common Aggregation Functions

Use functions to summarize each group.

- sum() – Total
- mean() – Average
- count() – Count
- min() – Minimum
- max() – Maximum

Category	Sales
A	100
B	150
A	200
B	120

3. Examples

Get useful insights with groupby!

```
df.groupby('Category')['Sales'].sum()

df.groupby('Category')['Sales'].mean()

df.groupby('Category').agg({
    'Sales': ['sum', 'mean'],
    'Profit': 'max'
})
```

You can also use multiple aggregations!

4. GroupBy with Multiple Columns

Group by more than one column.

```
df.groupby(['Region', 'Category'])
  ['Sales'].sum()
```

Region	Category	Sales
East	A	100
East	B	120
West	A	150
West	B	180

5. Advanced Operations

Try more powerful options!

- filter() – Filter groups
- transform() – Modify data
- apply() – Custom functions

These help us get deeper insights!

6. Real-World Use Cases

- Sales analysis by region
- Customer segmentation
- Performance reports
- Data dashboards



7. Key Takeaways

- GroupBy helps summarize data by groups.
- Use aggregation functions (sum, mean, count...).
- Combine with multiple columns for deeper analysis.
- Turn data into actionable insights!

Group. Aggregate. Discover!



Pivot Tables and Cross Tabulation

Summarize · Analyze · Discover Insights



1. What is a Pivot Table?

A Pivot Table is a powerful tool in Excel to summarize and analyze large datasets by rearranging data.

Sales	Region	Product
100	East	A
150	West	B
200	East	A
120	North	C

Quick Summary!

Region	Total Sales
East	300
West	150
North	120

2. What is Cross Tabulation?

Cross Tabulation (Crosstab) is a method to analyze the relationship between two categorical variables.

Gender	Purchase	
	Yes	No
Male	60	40
Female	70	30

Helps to find patterns and comparisons.

3. Key Features

- Summarize large data easily
- Perform multi-level analysis
- Create cross-tab reports
- Calculate totals, averages and percentages
- Visualize results with charts



4. How to Create a Pivot Table?

1. Select the data range.
2. Go to Insert → PivotTable.
3. Choose the location (New or Existing Worksheet).
4. Drag fields to Rows, Columns, and Values.
5. Get your summary instantly!

Simple Steps, Big Insights!

5. Example: Pivot Table

Sales by Region and Product

Region	Product A	Product B	Total
East	500	300	800
West	450	350	800
North	400	200	600
Total	1350	850	2200



6. Example: Cross Tabulation

Age Group vs Product Purchase

Age Group	Buy	Not Buy
18–25	70	30
26–35	50	50
36–45	30	70

Shows how two categories are related!



7. Benefits & Tips

Benefits:

- Save time & reduce complex data
- Find trends and patterns
- Make better decisions

Tips:

- Use filters and slicers
- Choose appropriate fields
- Keep your data clean

Pivot Tables & Cross Tabulation = Smart Data Analysis!

Data Visualization – A Brief matplotlib API Primer

Plot · Customize · Explore

Turn your data into meaningful visuals with Matplotlib!

Create stunning charts in just a few lines of Python!

matplotlib

1. What is Matplotlib?

Matplotlib is a popular Python library for creating high-quality 2D plots and visualizations.

- Simple & Powerful
- Highly Customizable
- Works with NumPy & Pandas

2. Basic Setup

Import and create a figure:

```
import matplotlib.pyplot as plt
```

Create a plot canvas:

```
plt.figure(figsize=(6,4))
```

3. Common Plots



4. Customization

Add labels, titles and style:

```
plt.plot(x, y,'o-', color='blue')
plt.title('My Plot')
plt.xlabel('X Axis')
plt.ylabel('Y Axis')
plt.grid(True)
plt.show()
```

5. Example: Line Plot

```
import matplotlib.pyplot as plt
x = [1, 2, 3,  4, 5]
y = [1, 4, 9,, 16, 25]
plt.plot (x, y, marker='o',
        color='green', linestyle='--')
plt.title('Simple Line Plot')
plt.xlabel('X')
plt.ylabel('Y')
plt.grid(True)
plt.show()
```



6. More Features

- Subplots – multiple plots in one figure
- Legends – describe data
- Colors & Styles – make it attractive
- Save Figures – save as PNG, JPG, etc.

Tip:

Use plt.tight_layout() for better spacing!

7. Example: Bar Chart

```
import matplotlib.pyplot as plt
categories = ['A', 'B', 'C']
values = [25,  40, 35]
plt.bar(categories, values,
        color=['skyblue','orange','green'])
plt.title('Sales by Category')
plt.xlabel('Category')
plt.ylabel('Sales')
plt.show()
```



8. Key Takeaways

- Matplotlib helps you visualize data easily.
- Use simple commands to create plots.
- Customize your charts with styles and labels.
- Explore different plot types for better insights.

Start Plotting with Matplotlib Today!

Statistical Visualization with Seaborn

Explore · Analyze · Visualize

Turn your statistical data into beautiful insights!

Seaborn makes statistical plots easy and attractive!

seaborn

1. What is Seaborn?

Seaborn is a Python library built on Matplotlib, designed for statistical data visualization.

- Beautiful Style
- Works with Pandas
- Great for Statistics

2. Installation

Install using pip:

```
pip install seaborn
```

Import it:

```
import seaborn as sns
import matplotlib.pyplot as plt
```

3. Key Features

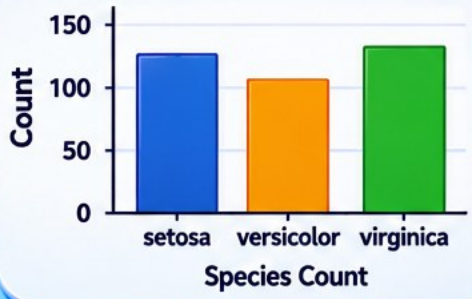
- Statistical plots
- Built-in themes
- Easy customization
- Works well with Pandas DataFrame

4. Popular Plots

- Count Plot
- Scatter Plot
- Box Plot
- Violin Plot
- Heatmap

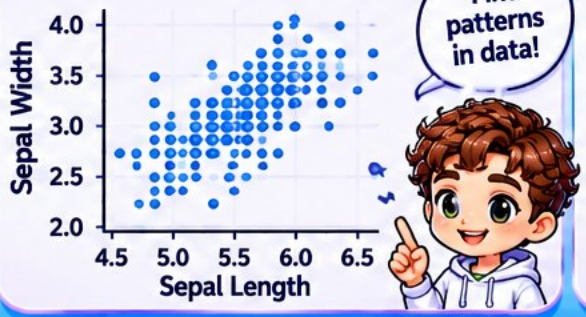
5. Example: Count Plot

```
sns.countplot(x='species',
              data=df)
plt.show()
```



6. Example: Scatter Plot

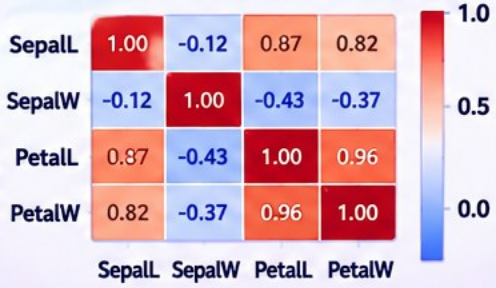
```
sns.scatterplot(x='sepal_length',
        y='sepal_width', data=df)
plt.show()
```



Find patterns in data!

7. Example: Heatmap

```
sns.heatmap(df.corr(),
        annot=True, cmap='coolwarm')
plt.show()
```



8. Summary & Tips

- Seaborn simplifies statistical visualization.
- Use different plot types for better analysis.
- Customize with themes and color palettes.
- Combine with Pandas for powerful insights.

Visualize Smarter with Seaborn!



Plotting Functions in pandas

Explore · Visualize · Analyze



1. What is Plotting in pandas?

Plotting in pandas helps us create graphs and charts directly from DataFrame or Series.



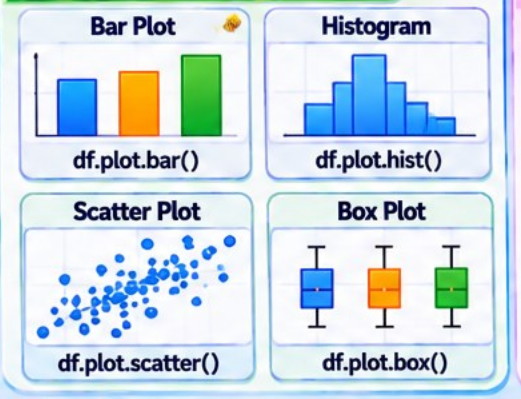
2. Basic Plot: Line Plot

Used for trends over time.

```
df.plot.line(x='Year', y='Sales')
```



3. Other Common Plots



4. Customizing Plots

Add labels, colors and style.

```
df.plot(kind='line', color='green',
        marker='o', title='Growth',
        xlabel='Year', ylabel='Value')
```



5. Plot Multiple Columns

Compare multiple features in one plot.

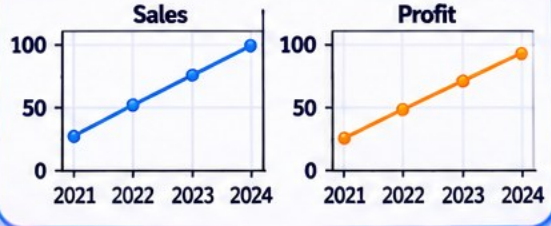
```
df[['Sales','Profit']].plot()
```



6. Subplots

Draw multiple plots together.

```
df[['Sales','Profit']].plot(subplots=True)
```



7. Style and Themes

Use different styles & themes.

```
plt.style.use('ggplot')
df.plot(kind='bar')
```



8. Key Takeaways

- Use df.plot() for quick plots.
- Try different plot types (line, bar, scatter, etc.).
- Customize with labels, colors and styles.
- Use subplots for multiple charts.
- Make your data come alive!



With pandas, plotting is simple and powerful!

Time Series
Real-World Case Study Applications

Analyze Trends · Forecast the Future · Make Better Decisions

Time Series helps us study data over time!

From stock prices to weather forecasts... Time Series is everywhere!

1. What is Time Series?

Time Series is data collected at regular time intervals (e.g., hourly, daily, monthly).

Examples:

Sales, Temperature, Stock Prices



2. Common Techniques

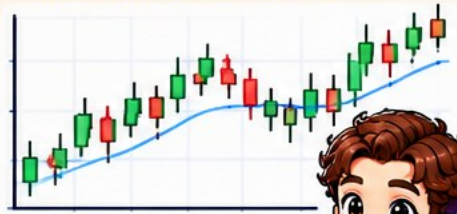
- Moving Average
- Exponential Smoothing
- ARIMA Models
- Seasonal Decomposition

3. Real-World Applications

- Stock Market Analysis
- Weather Forecasting
- Sales Forecasting
- Healthcare Monitoring

4. Case Study 1: Stock Market

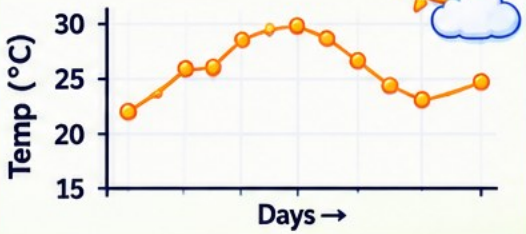
Use time series to predict stock prices and find trends.



- Analyze past prices
- Forecast future trends
- Plan investments

5. Case Study 2: Weather

Predict temperature using historical data.



- Find seasonal patterns
- Prepare for extreme weather

6. Case Study 3: Sales

Forecast future sales from past data.



- Plan inventory
- Increase business profit

7. Key Takeaways

- Time Series helps us understand trends and make forecasts.
- It is useful in finance, healthcare, business and more.
- With pandas, you can easily analyze time series data.

Time Series = Smarter Decisions!